



Introduction To Data Analytics

Data Analytics Bootcamp



What is Data Analytics?

- Data analytics is a way to find **useful patterns** hidden in large amounts of data.
- It solves problems by leveraging the power of **computation and statistics**.
- It helps people and companies make **fact-based decisions** instead of just guessing.



What Data Analytics Can Do

Reveal what
happened in the
past.

Guide the best
action to take
right now.

Predict what
might happen in
the future.

Transform difficult data into beautiful and digestible information.

How Data Analytics Works



Good questions

Organizing data

Identifying patterns

Visualizing findings

Continuously updating and refining...





How a Retail Store Can Use Data Analytics

1. "Which products are selling the best?"

The retail analytics process begins with this fundamental business question.

Store management needs to define what "best-selling" means - highest revenue, most units sold, highest profit margin, or fastest turnover.



How a Retail Store Can Use Data Analytics

2. Collect and clean sales data from all store locations

- Gather transaction data
- Clean the data by removing duplicates, correcting inconsistencies, and addressing missing values
- Standardize formats and ensure complete product details
- Create a unified, error-free dataset that's ready for analysis



How a Retail Store Can Use Data Analytics

3. Look at the data to find basic patterns

- Calculate key metrics: total units sold, revenue per product, profit margins
- Rank products by different performance indicators
- Identify top performers across various categories



How a Retail Store Can Use Data Analytics

4. Visualize data patterns for deeper insights

- Create interactive dashboards showing product performance metrics
- Generate time-series charts to illustrate seasonal trends
- Design customer segment visualizations linking buyer profiles to product preferences



How a Retail Store Can Use Data Analytics

5. Implement findings & continuously improve

- Adjust inventory and store layouts based on product performance data
- Develop targeted marketing campaigns and bundle offers for high-performing products
- Set up real-time monitoring dashboards with alerts for significant performance shifts
- Regularly refine analysis methods as new data becomes available

Continuously updating & refining



Why Data Analytics Matters

1. It helps **businesses** work better and save money.
2. It helps understand what **customers** really want.
3. It uses **facts** to make decisions instead of just guessing.
4. It can give companies an **advantage** over competitors.



Benefits Of Data Analytics

Enhanced Decision-Making

- Provides evidence-based foundations for critical decisions
- Eliminates subjective biases by relying on concrete data
- Enables executives, healthcare professionals, and policymakers to justify their choices with quantifiable metrics



Benefits Of Data Analytics

Operational Optimization

- Identifies inefficiencies and bottlenecks in organizational processes
- Streamlines workflows to maximize productivity
- Facilitates more effective resource allocation across departments



Benefits Of Data Analytics

Customer Understanding

- Analyzes consumer behavior patterns to reveal preferences and expectations
- Enables personalized marketing strategies and product recommendations
- Creates tailored customer experiences that increase satisfaction and loyalty



Topic

Coding As A Data Analyst



What Is Code?

Code is saying **one thing to mean something else**. In computer science, it refers to how we write out instructions.

Code is a tool used to **give the computer instructions** to later follow.

These instructions are called **programs**.



Programming Languages

Every program is written in a **language**:

- Anyone can create a computer language
- Different languages are good at different things

Computer languages are **deterministic**. Specific words mean specific things.



Thinking Programmatically

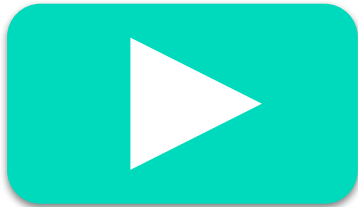
Humans think with guesses
and hunches.

Computers follow instructions
exactly as written.

- Break down steps small into specific actions rather than general concepts.
- Try to anticipate different scenarios your program might encounter - your computer won't be able to guess how to handle errors.



Acting Programmatically



While you're comfortable with the point-and-click world of **GUIs**, programming demands a different approach.



The very interface designed to make computers user-friendly becomes a **barrier** when writing code.

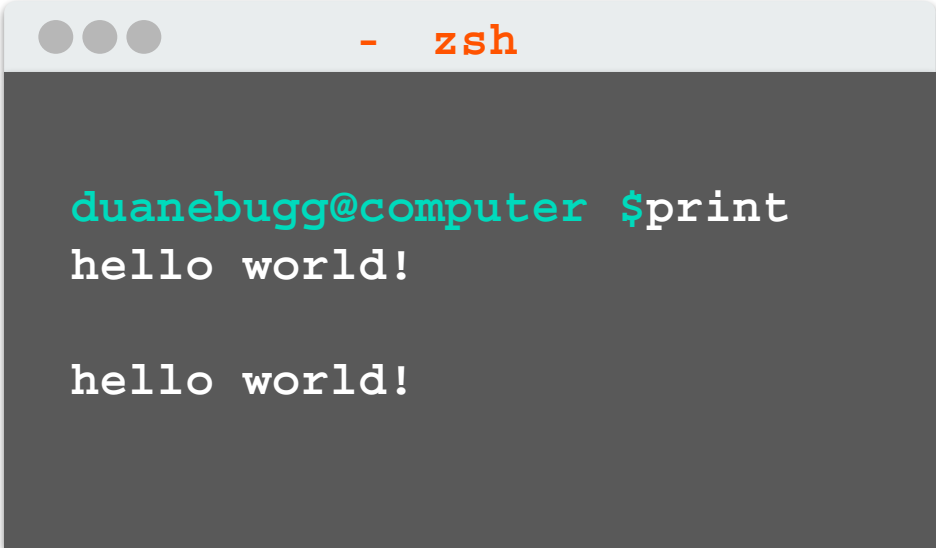


To act programmatically, you will need to become comfortable with the more efficient interface of **the terminal**.



The Terminal

We use the **Terminal** application to tell the computer commands in computer language.

A screenshot of a terminal window with a light gray title bar containing three window control buttons and the text '- zsh'. The terminal has a dark gray background. It shows a prompt 'duanebugg@computer' in teal, followed by the command '\$print' in white, and the output 'hello world!' on two separate lines in white.

```
duanebugg@computer $print  
hello world!  
  
hello world!
```

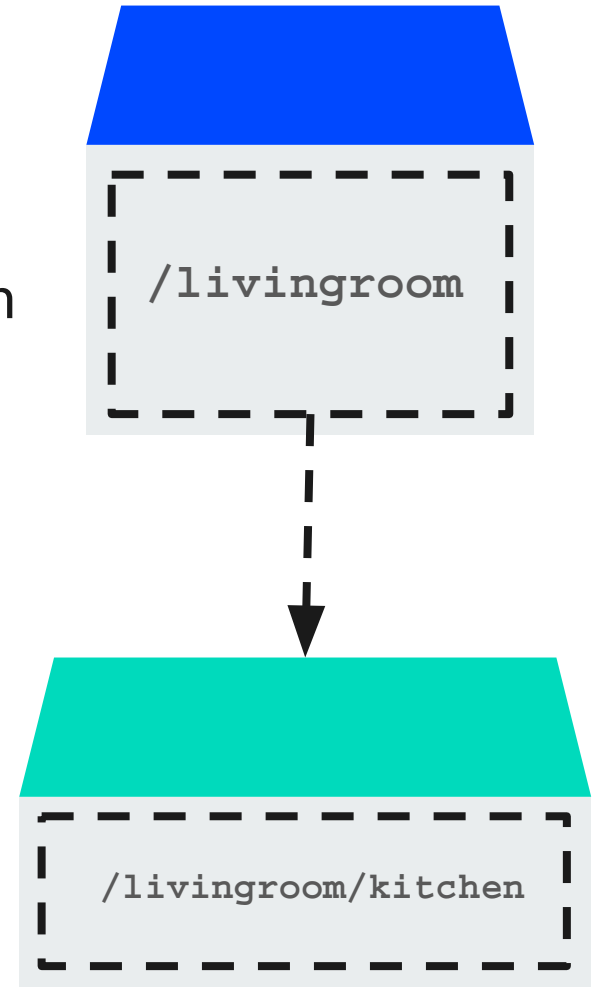
- The terminal gives us a **command line** that is managed by a **shell**.
- *command line, console, shell, and terminal* are often used interchangeably.

Location & Navigation

The terminal is always referencing a directory. This directory is “your” location on the computer through the terminal.

Navigating your computer is like exploring rooms in a house.

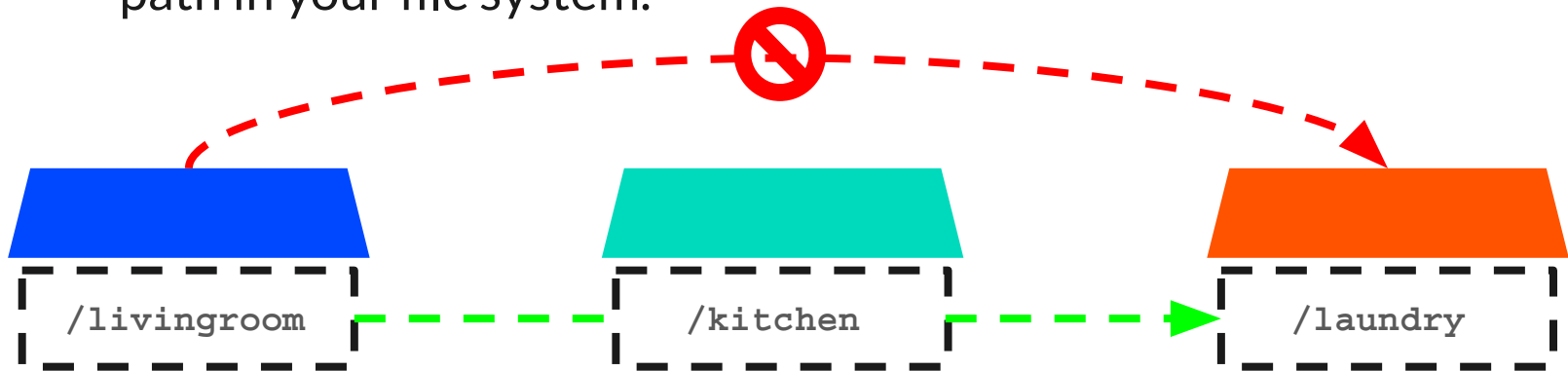
- The terminal can always tell you exactly where it's at.
 - Type **pwd** or **cd** and hit the Enter/Return key.



Directories & Paths

Directories are locations on your computer that you will visit via the terminal. These are your “rooms”.

- Like rooms, there are specific **paths** to reach them.
- If the laundry only has a door from the kitchen, you can't get there from the living room.
- `/livingroom/kitchen/laundry` would be the **correct** path in your file system.





Basic Path Navigation

"Print Working Directory"

- **Mac** Command: **pwd**
- **Windows** Command: **cd** (with no arguments)
 - Shows the name of the current directory.

"List"

- **Mac** Command: **ls**
- **Windows** Command: **dir**
 - Shows the contents of the current directory

"Change Directory"

- **Mac & Windows** Command: **cd** [directory name]
 - Shows the contents of the current directory



Topic

Git/Hub Refresh



What Is Git?

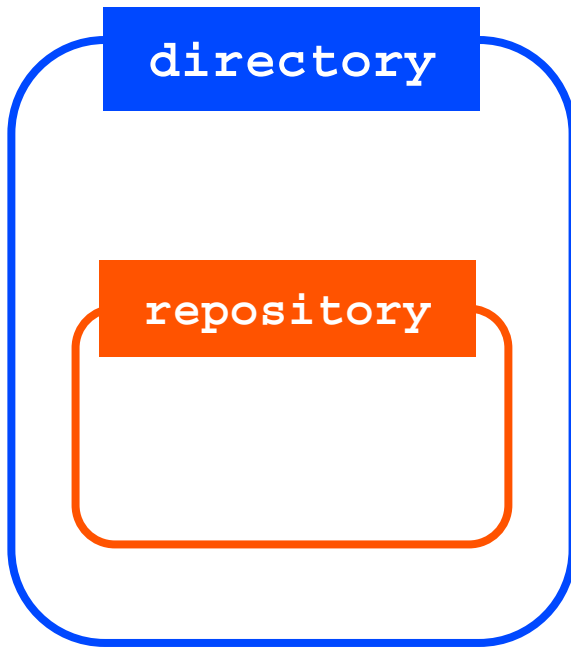
- **git** is a distributed version control tool.
- Developer teams use it to keep track of project changes and switch between different versions.
- We can track the following:
 - A log of changes to the files
 - Including who made each change
 - And why they made that change



Git VS. GitHub

- **GitHub** is a hub for software versions managed using **git**.
- You can think of it as a social network (**Hub**) where people share their code (**git**).

Git Repository

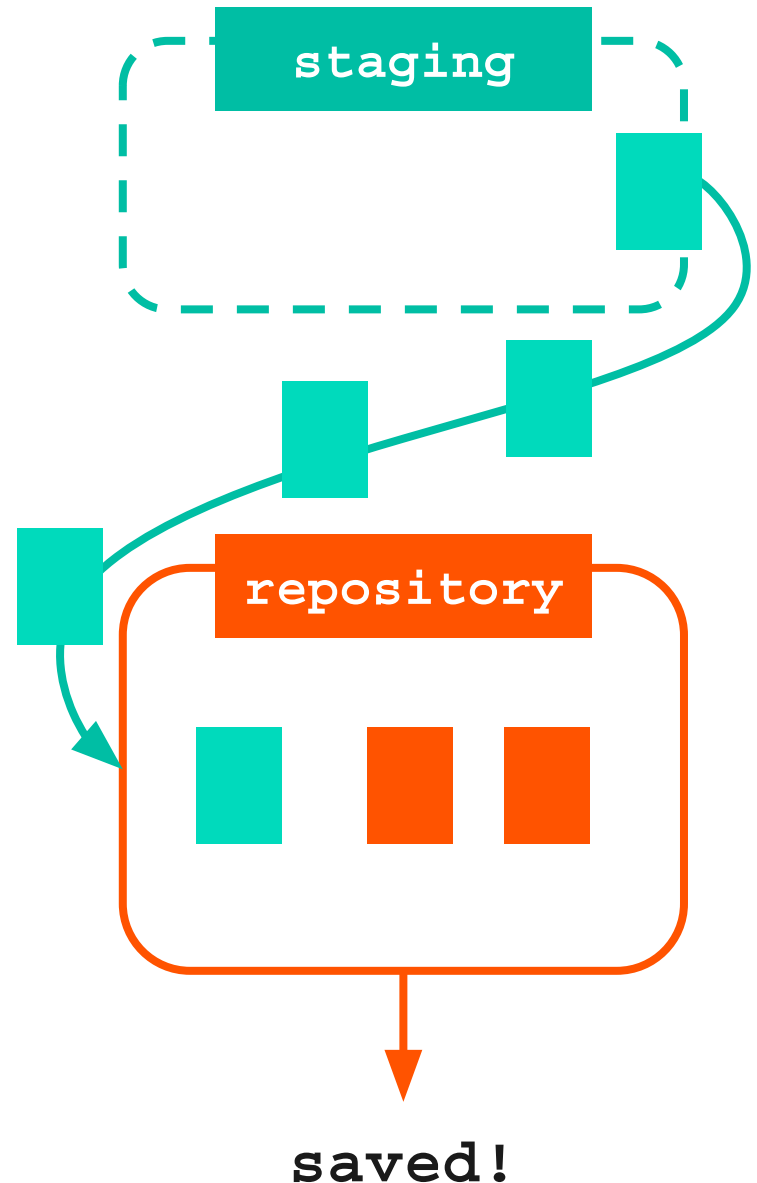


A repository (or "repo") contains all of the versions of a project. Git repositories and your directories are **separate** - the repository lives inside of the directory.

- **git init** (short for initialize) makes the current directory a repository that you can now track.
- **git status** lists the changes that have been tracked in the repository.

Saving Changes

- git has a two-step process for (“saving”) tracking changes to files.
 - First, you **add** the changes to the staging area (also called the cache or the index)
 - Then, you **commit** the changes to the history (also called the log)
- After a commit, the staging area is cleared, and the cycle continues.





Commit Messages

When committing, you always provide a message describing why the changes are there (not a summary).

- `git commit -m "showing correlation between age and purchase frequency"`

Without commit messages, you will be tracking **what** you changed, but not **why** you made the changes.



Basic Git/Hub Commands

"Push Changes to Remote"

- Command: `git push`
- Command with arguments: `git push origin main`
 - Uploads your committed changes to the remote repository.

"Pull Changes from Remote"

- Command: `git pull`
- Command with arguments: `git pull origin main`
 - Downloads changes from the remote repository to your local repository.



Exercise

Terminal Platform



Topic

Python & Jupyter Notebooks



What Is Python?

Python is a programming language commonly used in data analytics. While there are other languages used, this program uses Python because of its:

- Simple, readable syntax
- Extensive standard library
- Large and active community
- Versatility across different fields (web development, data science, AI)



“What about R? SAS? Julia? MATLAB?”

Learn One, Learn Any!

- Core concepts (variables, loops, functions) exist in all languages.
- Problem-solving mindset is universal.
- Syntax differences are just "accents" of the same thinking.



“Where do we write Python?”

Jupyter Notebook is an **IDE** (Interactive Development Environment) that combines:

- Live code
- Rich text (Markdown)
- Visualizations
- Equations
- Media



Jupyter Notebooks: Key Features

1. Web-based interface
2. Combines code and documentation
3. Code executes in "cells"
4. Results display immediately
5. Great for data analysis and learning
6. File extension: .ipynb



Notebook Structure

A notebook consists of different types of cells:

- **Code cells:** For writing and running code
- **Markdown cells:** For formatted text and documentation
- **Raw cells:** For unformatted plain text
- **Output cells:** Display results automatically



Code & Markdown Cells

Code Cells

Where you write and execute Python code.

- Execute with Shift + Enter
- Output appears below the cell
- State is shared between cells
- Can be re-run in any order

Markdown Cells

For documentation and explanations.

- Headers (#, ##, ###)
- Lists (* or numbers)
- Bold (**text**)
- Italic (*text*)
- Code blocks (`python`)

Markdown Cell

Analyzing Sales Data

Let's calculate the monthly sales growth rate from our dataset.

The formula we'll use is:

$$\text{Growth Rate} = \frac{\text{Current Month} - \text{Previous Month}}{\text{Previous Month}} \times 100\%$$

First, we'll import the necessary libraries and calculate the month-over-month growth rates:

Code Cell

```
import pandas as pd
import matplotlib.pyplot as plt

# Load and process sales data
sales_data = pd.read_csv('monthly_sales.csv')
sales_data['Growth_Rate'] = sales_data['Sales'].pct_change() * 100

# Plot growth rates
plt.bar(sales_data['Month'], sales_data['Growth_Rate'].fillna(0))
plt.title('Monthly Sales Growth Rate')
plt.ylabel('Growth Rate (%)')
plt.show()
```



Jupyter Notebooks: **Best Practices**

1. Use clear titles and headers
2. Document your code with markdown
3. Keep cells focused and concise
4. Restart and run all periodically
5. Clear output before sharing
6. Create logical flow of analysis



“Why Jupyter Notebooks?”

Jupyter Notebooks are **ideal** for:

1. Scientific computing
2. Creating interactive reports
3. Prototyping code
4. Sharing research findings
- 5. Data analysis**



Exercise

Running A Jupyter Notebook